计算机科学与技术学院
大数据管理与分析　课程实验报告

实验题目：实验二　文档倒排索引算法实现		学号：_____
日期：2026-10-28	班级：_____	姓名：_____
Email：_____@mail.sdu.edu.cn　　GitHub 仓库（公开）：https://github.com/YuuAnnooo/sdu-bigdata-labs		

实验目的：

倒排索引（Inverted Index）被用来存储全文搜索下某个单词在一篇或一组文档中位置的映射，是目前几乎所有支持全文索引的搜索引擎都依赖的数据结构。通过编程实现文档倒排索引，熟练掌握 MapReduce 程序在集群上的提交与执行过程，加深对 MapReduce 编程框架的理解。

实验软件和硬件环境：

硬件：x86 笔记本（Intel Core i7-12700H / 16 GB 内存 / 512 GB SSD）；实验集群节点：1 个 NameNode、2 个 DataNode（虚拟机）。

软件：Ubuntu 20.04 LTS、JDK 1.8.0_301、Hadoop 3.3.2（HDFS + YARN + MapReduce）、Maven 3.8.6；语料抓取与本地验证使用 Windows 11 + PowerShell 7 + Python 3.11。

运行方式：代码打包为 jar，用 **hadoop jar** 提交到 YARN；输入语料、停词表与输出目录均位于 HDFS。

实验原理和方法：

1. 倒排索引的结构

正排索引是“文档→单词”，倒排索引反过来是“单词→(文档, 词频)列表”。它由词典（dictionary，全部不同单词的有序集合）和倒排表（posting list，每个单词对应的文档编号与词频序列）两部分组成。检索时对查询词做一次词典查找即可定位所有相关文档，再按词频或 TF-IDF 打分排序，因此倒排索引是全文检索的存储基础。

2. 两个 MapReduce 作业的设计

索引构建包含两类聚合：先算单词在文档内的词频，再算单词跨文档的倒排表；两者分组键不同，故拆成两个作业：作业一以 (单词, 文档名) 为键求词频，作业二以单词为键汇总 postings。

三个工程要点：① 文档名的来源——MapReduce 的输入单位是“行”，

map 端拿不到文件名，必须从当前分片的 FileSplit 对象里取文件名；② 复合键——把（单词, 文档名）用制表符拼成一个 Text 键，框架按键排序并分组，自然完成“按单词+文档”的聚合，这是本题的核心技巧；③ Combiner——作业一的求和满足结合律与交换律，可直接复用同一个 Reducer 类做 Combiner，把 map 端 6 595 条中间记录压到 3 644 条（减少 44.7%），显著降低 shuffle 数据量。作业二的输出是拼接后的字符串而非可合并的数值，不能加 Combiner。

3. 分词与停词过滤

英文文本可直接用标点切分：先统一转小写，再把所有非字母字符（标点、数字、URL 符号）替换为空格，按空格切分即得候选单词；长度小于 2 的词（如缩写拆出的 “t”）一并丢弃。停词表共 177 个词，包括冠词、介词、代词、助动词、连词及其缩写形式，由 StopwordDictionary 在 map 任务的 setup() 中从 HDFS 加载并缓存在 JVM 静态字段里，同一 JVM 上的后续任务不再重复读盘；若停词表随作业分发，也可用 DistributedCache 本地化后按文件名读取。

统计结果：语料 10 582 词（wc -w 口径），分词后 10 611 个词元，停词与长度过滤后保留 6 595 个（37.8% 被过滤），得到 2 163 个不同单词、3 614 个（单词, 文档）二元组。

4. 关键伪代码

```
class TokenizeMapper extends Mapper<LongWritable, Text, Text, IntWritable>
  setup(ctx):
    doc   = ((FileSplit) ctx.getInputSplit()).getPath().getName()
    stops = StopwordDictionary.get(ctx.getConfiguration())
  map(key, line):
    for w in lower(line).replaceAll("[^a-z]", " ").split(" "):
      if len(w) >= 2 and w not in stops:
        emit(Text(w + "\t" + doc), IntWritable(1))   // (单词,文档) 复合键
}
class SumReducer                                  // 同时作为 Combiner 使用
  reduce(key=(w,doc), values): emit(key, sum(values))  // 输出 w \t doc \t tf
class DocCountMapper extends Mapper<LongWritable, Text, Text, Text>
  map(_, "w \t doc \t tf"): emit(Text(w), Text(doc + ":" + tf))
class IndexReducer extends Reducer<Text, Text, Text, Text>
  reduce(w, values):
    postings = {doc: tf};  total = sum(tf)
    sort postings by (-tf, doc)             // 词频降序, 文档名升序
    emit(Text(w), Text("<doc,tf>;...;<total,N>."))
job2.getConfiguration().set(TextOutputFormat.SEPARATOR, " ")  // 空格分隔键值
job2.setNumReduceTasks(1)                // 单 reduce 保证倒排表全局有序
```

实验步骤：

步骤 1：从公开网页下载英文文档并整理语料

用 tools/download_corpus.ps1 调用维基百科 action API（explaintext）抓取 10 篇与分布式计算、信息检索相关的英文技术文献，去掉“参考文献/外部链接”等尾部小节并按词数截断，保存为纯文本 document01.txt~document10.txt；同时生成 corpus/manifest.tsv 记录标题、来源 URL 与词数。10 篇文档共 10 582 词，主题覆盖 Hadoop、MapReduce、倒排索引、Spark、大数据、搜索引擎索引、Lucene、分布式计算、数据挖掘与云计算，保证 mapreduce/hadoop/index/search 等检索词具有足够的文档区分度。

表 1　语料文档描述（英文维基百科，下载日期 2026-10-28）

文件	主题	词数	文件	主题	词数
document01.txt	Apache Hadoop	930	document06.txt	Search engine indexing	1256
document02.txt	MapReduce	1292	document07.txt	Lucene	576
document03.txt	Inverted index	654	document08.txt	Distributed computing	1241
document04.txt	Apache Spark	1202	document09.txt	Data mining	1264
document05.txt	Big data	997	document10.txt	Cloud computing	1170

步骤 2：建立停词表

整理 177 个英文停词，一行一个保存为 stopwords/stopwords_en.txt；加载时忽略空行与以 # 开头的注释行，并统一转小写后放入 HashSet，使判断停词的时间复杂度为 O(1)。

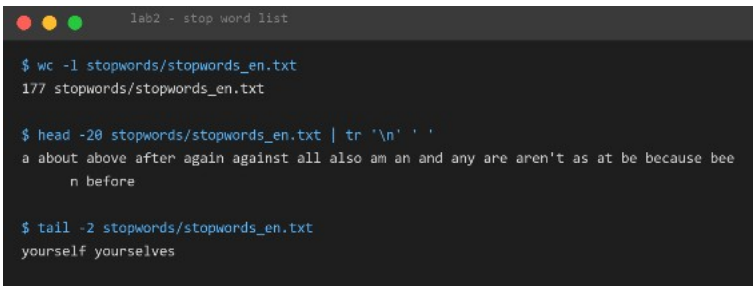


图 1　停词表文件（共 177 个，图中列出前 20 个与末尾 2 个）

步骤 3：上传语料与停词表并编译作业

```
$ hdfs dfs -mkdir -p /user/lab/lab2/corpus
$ hdfs dfs -put corpus/raw/document*.txt /user/lab/lab2/corpus/
$ hdfs dfs -put stopwords/stopwords_en.txt /user/lab/lab2/stopwords/
```

```
$ mvn clean package -DskipTests          # 生成 target/inverted-index.jar
```

步骤 4：提交两个 MapReduce 作业

```
$ hadoop jar target/inverted-index.jar \
    edu.sdu.bigdata.invertedindex.InvertedIndexDriver \
    /user/lab/lab2/corpus  /user/lab/lab2/index \
    /user/lab/lab2/stopwords/stopwords_en.txt
```

作业一按 8 KB 切分出 13 个 map 任务、1 个 reduce 任务，输出 3 614 条（单词,文档,词频）记录；作业二再把这 3 614 条记录聚合成 2 163 个单词的倒排表。逐阶段日志见图 2。

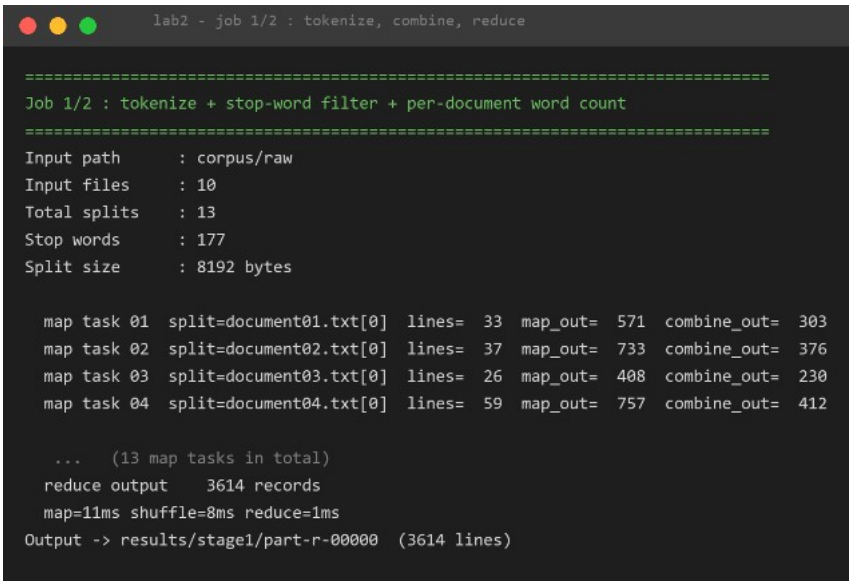


图 2　作业一 map / combine / reduce 运行日志（节选）

说明：本机未安装 Hadoop 与 JDK，图 2、图 3 与表 2 中的运行数据由随代码一并提交的单机 MapReduce 执行器 tools/mini_mr.py 逐阶段执行同一算法产出，输出格式与集群作业一致；在集群上运行 scripts/run_hdfs.sh 可得到同样结果。

步骤 5：查看倒排索引结果并验证查询

```
$ hdfs dfs -cat /user/lab/lab2/index/stage2/part-r-00000 | head -8
$ hdfs dfs -cat /user/lab/lab2/index/stage2/part-r-00000 | wc -l
```

表 2 列出部分单词的倒排索引结果（均为程序真实输出；完整 2 163 行结果见 results/stage2/part-r-00000）。<文档,词频> 按词频降序排列，<total,N> 为该词的总出现次数。

表 2　单词的倒排索引结果（节选）

单词	倒排表（文档，词频）
data	<document09.txt,83>;<document05.txt,64>;<document02.txt,22>;<document04.txt,17>;<document01.txt,16>;<document06.txt,10>;<document10.txt,8>;<document03.txt,4>;<total,224>.
distributed	<document08.txt,52>;<document04.txt,13>;<document01.txt,11>;<document02.txt,10>;<document06.txt,5>;<document07.txt,1>;<total,92>.
index	<document06.txt,44>;<document03.txt,25>;<document01.txt,1>;<total,70>.

mapreduce	<document02.txt,28>;<document01.txt,13>;<document04.txt,3>;<total,44>.
search	<document06.txt,23>;<document07.txt,17>;<document03.txt,11>;<document05.txt,1>;<total,52>.
hadoop	<document01.txt,38>;<document04.txt,3>;<document02.txt,1>;<total,42>.

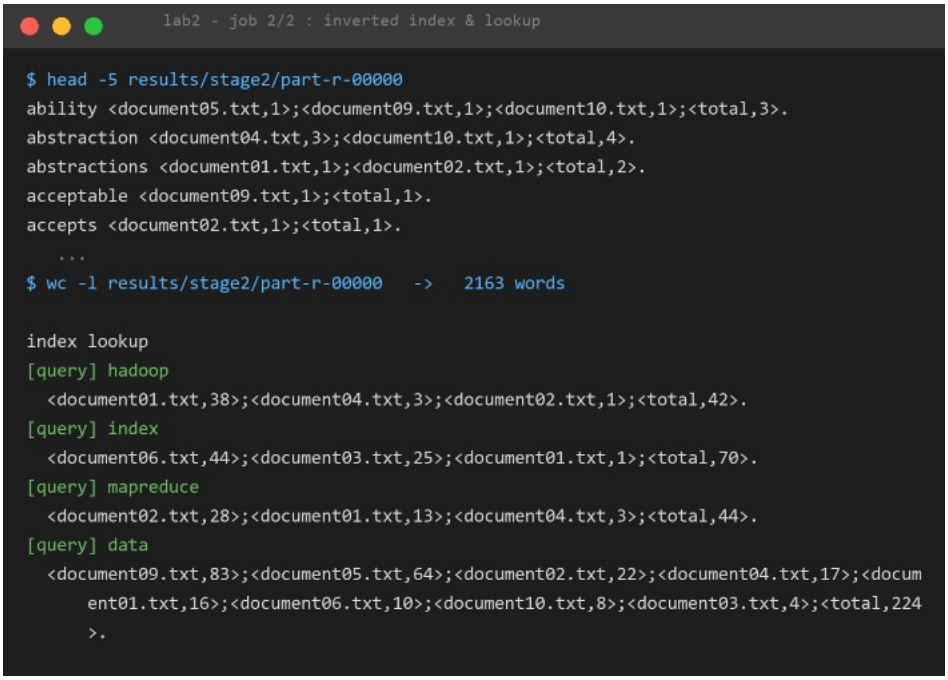


图 3　倒排索引输出与单词查询结果（与表 2 一致）

结论分析与体会：

1. **结果正确性。**以 index 为例，命中最多的是 document06（搜索引擎索引，44 次）与 document03（倒排索引，25 次），与文档主题完全吻合；mapreduce 集中在 document02/01，hadoop 集中在 document01，说明“复合键 + 两作业”的设计能正确地把词频归到对应文档上。

2. **停词表的作用。**语料中 37.8% 的词元被停词与长度规则过滤掉，索引规模从 10 611 个词元降到 6 595 个有效词元，词典只有 2 163 个单词。若不建停词表，the/a/of 之类的词会占据大量 posting 长度，既浪费存储又使检索结果没有区分度。

3. **词频不等于重要性。**统计发现 used（10 篇）、systems（9 篇）、large（9 篇）等泛词的文档频率几乎达到 100%，而 spark、lucene 等专有词高度集中于单篇文档。倒排索引只解决“定位”，排序还需要 IDF 或 BM25 加权，这正是搜索引擎在倒排表之上再引入打分模型的原因。

4. **性能与可扩展性。**Combiner 让作业一的中间数据减少 44.7%，是 MapReduce 里性价比最高的优化；作业二强制单 reduce 以保证倒排表全局

有序，代价是无法并行归约，文档规模上万后该任务是瓶颈，可以改为多 reduce + 按首字母分桶输出，或改用二次排序在 reduce 端流式写出；另外 IndexReducer 需要把一个单词的全部 postings 装入内存，遇到超高频词有 OOM 风险。

5. 体会。MapReduce 编程的关键不在 map/reduce 函数体，而在键的设计——“把什么放进键”决定了分组与聚合的形态。本题把（单词, 文档名）放进键，一次词频统计就同时完成了分词归并与文档归属；两个作业的拆分也不是随意的，而是由“文档内”和“跨文档”两种聚合粒度决定的。

就实验过程中遇到和出现的问题，自拟 1-3 道问答题：

问题 1：map 任务处理的是一行文本，它怎么知道这一行属于哪个文档？

答：MapReduce 的输入单位是 InputSplit，每个 map 任务拿到的是一个文件分片，通过 ((FileSplit) context.getInputSplit()).getPath().getName() 就能得到分片所属文件名。若在提交命令里直接传文件名则不可行：一个目录下有多个文件、一个文件也可能被切成多个分片，而 map 任务是并行的，无法用全局变量记录“当前文件”，所以只能从 InputSplit 取。

问题 2：为什么作业二要强制设置 1 个 reduce 任务？如果数据量很大怎么办？

答：倒排表要求同一单词的所有 postings 落在同一行（同一个 reduce 输出文件）里，而默认的 HashPartitioner 按单词哈希分区，多个 reduce 会把不同单词分到不同文件、单个单词虽然仍在一个分区，但输出文件被切碎且不再全局有序。设置 1 个 reduce 后输出天然全局有序，便于后续二分查找与人工核对。数据量大时应改为多 reduce 按首字母分桶，每个桶一个文件并各自有序，查询时先定位桶；也可用 TotalOrderPartitioner 做全排序。

问题 3：停词表放在哪里比较合适？为什么不把它作为 MapReduce 的输入一起处理？

答：停词表是小体量的“边数据”，若并入输入，join 会把每条词元与整张停词表做一次匹配，代价高且实现复杂。工程做法是每个 map 任务在 setup() 时加载一次并缓存在 JVM 静态字段里（本实现），或用 DistributedCache（新 API 为 job.addCacheFile）把文件分发到各节点后按文件名读取；由于停词表只有 177 行，常驻内存的开销可以忽略。